Functional & Topological Statistics: Topological Persistence in Image Segmentation

Luca Simonelli

March 21, 2026

Image Segmentation

A prominent challenge in the field of image processing is the *Segmentation Problem*. This task is formally defined as the identification of underlying objects or boundaries within a digital image, with the objective of partitioning it into a set of semantically meaningful and coherent regions.

Mathematically, a 2-dimensional grayscale image can be modeled as a function $f : D \subset \mathbb{R}^2 \to \mathbb{R}$, which is typically piecewise constant on a rectangular grid of pixels. In this framework, the function value at a specific pixel represents the grayscale intensity. For RGB images, this extends to a vector-valued function $f : D \to \mathbb{R}^3$, representing the intensity of the three primary color channels.

The segmentation problem is often considered "ill-defined" for two main reasons:

- Defining "regions of importance" in a strictly mathematical sense is non-trivial and context-dependent.
- Every image presents unique noise patterns and structural complexities, requiring adaptive approaches rather than a single universal algorithm.

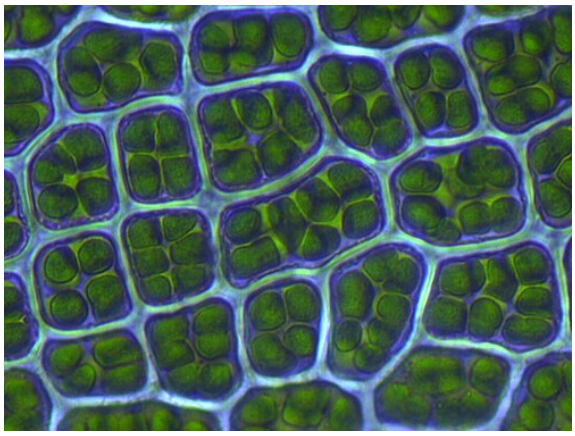


Figure 1: Microscopy of *Mnium hornum* (Laminaria) cells.

A practical application involves detecting cellular organelles, such as chloroplasts, within the image shown in Figure 1. A standard approach for this task is the **Watershed Algorithm**. However, this method suffers from a significant drawback: *over-segmentation*. Due to its sensitivity to local extrema, the algorithm often identifies too many features, creating a fragmented output when the goal is simply to outline major boundaries or contours.

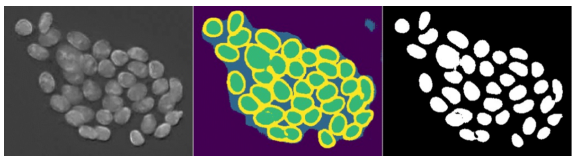


Figure 2: Example of cell nuclei segmentation showing the challenge of overlapping structures.

Theoretical Framework: Morse Theory

To refine the Watershed algorithm using topological persistence, we must model the image as a smooth landscape. This is achieved through *Morse Theory*, which studies the relationship between the topology of a manifold and the critical points of a function defined on it.

Morse Functions and Critical Points

Let M be a smooth manifold (in our case, the image domain $D \subset \mathbb{R}^2$). A smooth function $f : M \to \mathbb{R}$ is called a **Morse function** if all its critical points are non-degenerate.

A point $p \in M$ is a **critical point** if the gradient vanishes: $\nabla f(p) = 0$. Non-degeneracy implies that the *Hessian matrix* $H_f(p)$ is non-singular (its determinant is non-zero).

The **index** of a critical point p is the number of negative eigenvalues of $H_f(p)$. In 2D image processing:

- **Index 0:** Local minimum (a "valley" where a region starts).
- **Index 1:** Saddle point (a "pass" where two regions meet).
- **Index 2:** Local maximum (a "peak").

The Morse Lemma

The behavior of a Morse function near a critical point is strictly regulated by the **Morse Lemma**, which states that there exists a local coordinate system $(x_1, \ldots, x_n)$ such that:

$$f(x) = f(p) - x_1^2 - \cdots - x_\lambda^2 + x_{\lambda+1}^2 + \cdots + x_n^2 \tag{1}$$

where λ is the index of the critical point. This ensures that the topological changes in the image level sets only occur at these discrete critical points.

Stable and Unstable Manifolds

The dynamics of the Watershed algorithm are governed by the gradient flow lines. For a critical point p, we define:

- **Stable Manifold $W^s(p)$:** The set of points that flow into p under gradient descent.
- **Unstable Manifold $W^u(p)$:** The set of points that originate from p.

In the context of the Watershed, the *catchment basins* correspond to the stable manifolds of the local minima. The *watershed lines* (the boundaries) are formed by the unstable manifolds of the saddle points that separate these basins.

Integration with Persistence

The connection between Morse Theory and Persistence lies in the **Persistence Pairing**. Topological persistence identifies pairs of critical points—specifically a minimum (index 0) and a saddle (index 1)—that are "topologically close."

If the difference in their functional values $f(p_{\text{saddle}}) - f(p_{\text{min}})$ is small, the pair is considered noise. We use this to:

1. **Simplify the function:** By "canceling" critical pairs with low persistence, we effectively flatten insignificant valleys.
2. **Seed the Algorithm:** Only minima that "survive" a certain persistence threshold τ are used as markers, effectively solving the over-segmentation problem.

Foundations of Simplicial Topology

Simplices and Vertices

A k**-simplex** σ is the convex hull of $k+1$ affinely independent points $\{v_0, v_1, \ldots, v_k\}$ in $\mathbb{R}^n$:

- A **0-simplex** is a **vertex** (a single pixel).
- A **1-simplex** is an **edge** (adjacency between two pixels).
- A **2-simplex** is a **triangle** (the filled area between three mutually adjacent pixels).

Simplicial Complexes

A **simplicial complex** K is a finite collection of simplices such that:

1. Every face of a simplex in K is also in K.
2. The intersection of any two simplices $\sigma_1, \sigma_2 \in K$ is either empty or a common face of both.

Lower Star Filtration

Given a PL function f defined on the vertices of K, the **lower star** of a vertex v is:

$$St_{-}(v) = \{\sigma \in K : \max_{u \in \sigma} f(u) = f(v)\} \tag{2}$$

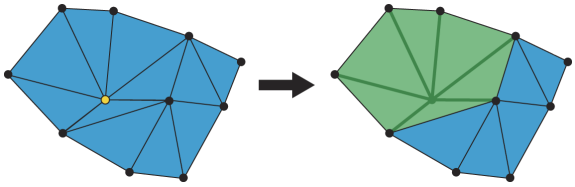


Figure 3: Star complex.

By adding lower stars in increasing order of $f(v)$, we build a **filtration** $K_0 \subseteq K_1 \subseteq \cdots \subseteq K_n = K$. The Watershed algorithm tracks the birth and death of connected components (H_0 homology) across this filtration.

From Smooth to Discrete: Piecewise Linear (PL) Functions

In practical image processing, we do not deal with smooth manifolds but with discrete grids of pixels. To bridge this gap, we represent the image as a **Piecewise Linear (PL) function** defined on a simplicial complex K (typically a triangulation of the pixel grid).

A PL function $f : |K| \to \mathbb{R}$ is uniquely determined by its values at the vertices V of the complex. For any point x within a simplex, the value $f(x)$ is obtained via linear interpolation of the values at the vertices.

Discrete Critical Points

In the PL setting, the concept of a critical point is replaced by the study of the **lower link** of a vertex v:

$$Lk_{-}(v) = \{\sigma \in Lk(v) : \forall\, u \in \sigma,\ f(u) \leq f(v)\} \tag{3}$$

The topology of $Lk_{-}(v)$ determines the nature of the vertex:

- **Local Minimum:** $Lk_{-}(v)$ is empty.
- **Saddle Point:** $Lk_{-}(v)$ is disconnected (two or more components).
- **Regular Point:** $Lk_{-}(v)$ is contractible (e.g., a single arc).

Connection to Morse Theory

In the context of the Watershed algorithm, we treat the pixel grid as a graph. The PL approach allows us to:

1. **Order the pixels:** Sort vertices by intensity to simulate the flooding process (*Lower Star Filtration*).
2. **Track Topology:** As vertices are added in increasing order of $f(v)$, monitor the birth and death of connected components (H_0 homology).
3. **Identify Merges:** A saddle point in the PL sense is exactly where two catchment basins would meet in the continuous Watershed.

Intuition Behind the Watershed Algorithm

Before formalizing the algorithm, it is useful to build a geometric intuition through a series of increasingly detailed steps.

Step 1 — The Image as a Landscape

A grayscale image $f : D \to [0, 1]$ can be interpreted as a topographic surface: the intensity of each pixel equals the *altitude* of the corresponding point of the terrain. Three types of critical points play a key role (Figure 4):

- **Local minima** — valleys where water would collect.
- **Saddle points** — mountain passes where two valleys nearly meet.
- **Local maxima** — peaks (less relevant for segmentation).

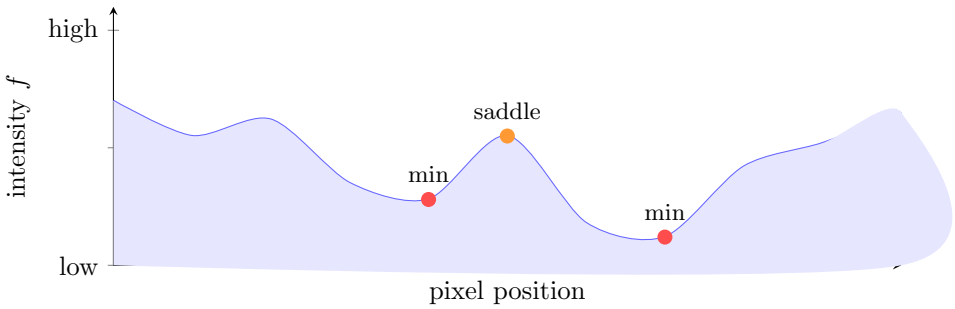


Figure 4: A 1-D slice of an image interpreted as a landscape. Red circles mark local minima; the orange circle marks a saddle point.

Step 2 — The Flooding Process

Imagine piercing the terrain at each local minimum and slowly immersing the entire landscape in water. As the water level rises, **catchment basins** fill up independently around each minimum (Figure 5).

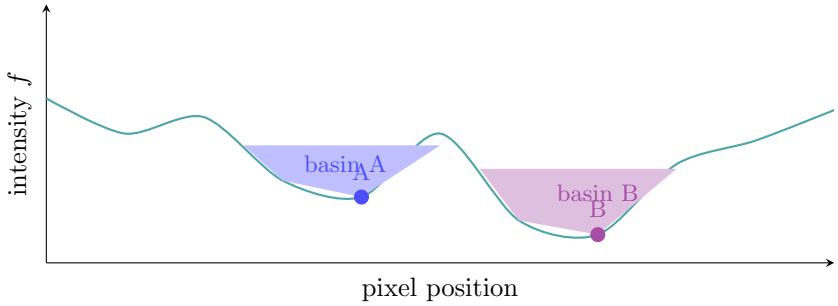


Figure 5: Two catchment basins filling independently from their respective minima.

Step 3 — Building the Dams

Eventually, the rising water from two adjacent basins meets at a saddle point. To prevent the basins from merging, a **watershed line** (dam) is erected at that junction (Figure 6).

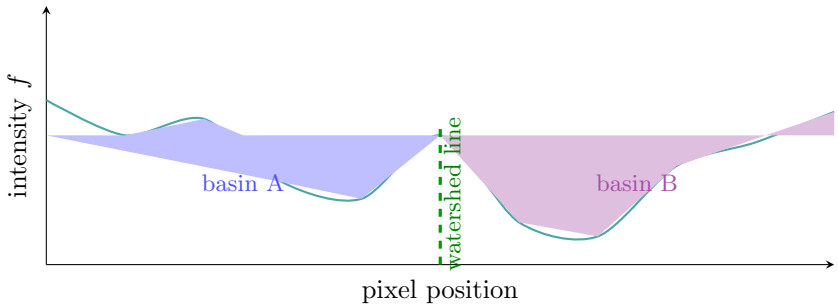


Figure 6: A watershed line (green dashed) erected at the saddle point where basins A and B would otherwise merge.

Step 4 — The Over-Segmentation Problem

The standard Watershed treats every local minimum as a seed, including those caused by image noise. A noisy image may contain hundreds of spurious minima, each generating an unwanted basin (Figure 7).



Figure 7: Noise introduces many spurious local minima (red dots), each seeding an unwanted catchment basin.

Step 5 — Topological Persistence as a Filter

For each local minimum $v_{\min}$ and the saddle s at which it merges with an older basin, the **persistence** is:

$$p = f(s) - f(v_{\min}).$$

A small p means the basin is shallow (likely noise); a large p means the basin is deep (a genuine feature). By choosing a threshold τ, we retain only the minima with $p \geq \tau$ (Figure 8):

$$\text{surviving minima} = \{v_{\min} : f(s) - f(v_{\min}) \geq \tau\}.$$

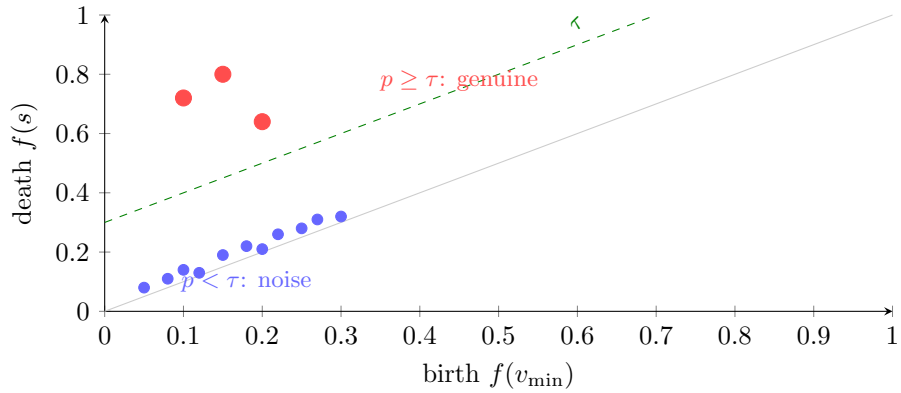


Figure 8: Persistence diagram. Points far above the diagonal (red) have high persistence and correspond to real features; points near the diagonal (blue) are noise. The dashed line marks the threshold τ.

Step 6 — Seeded Watershed: the Final Result

Only the surviving minima are used as **seeds**. A priority-queue BFS is run from each seed, expanding basins in order of increasing f (Figure 9):

1. Each seed is inserted into a min-heap ordered by f.
2. The pixel with the smallest f is extracted and its unlabelled 8-connected neighbours are assigned the same label.
3. The process terminates when every pixel has been assigned.

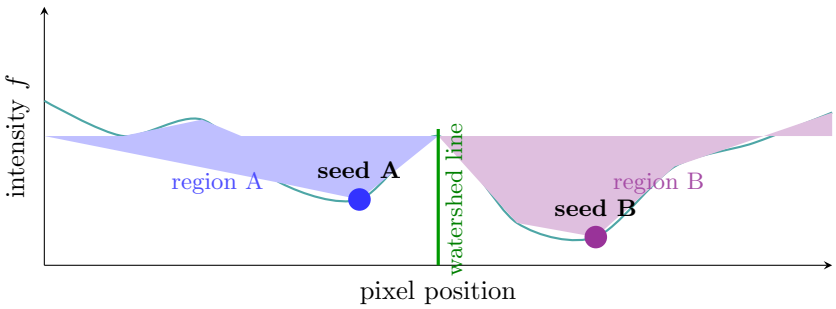


Figure 9: Seeded watershed: only two persistent minima survive as seeds, producing two clean regions.

The complete pipeline is summarised in Figure 10.

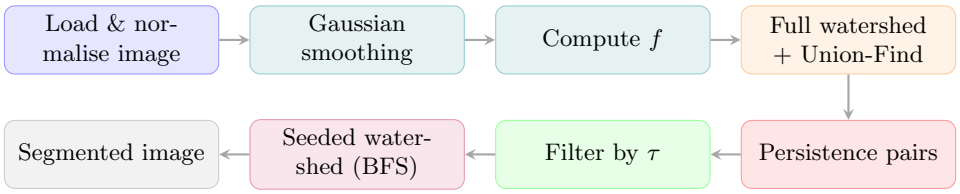


Figure 10: Full pipeline: from raw image to segmented output.

The Watershed Algorithm

The Watershed algorithm is a powerful morphological tool for image segmentation. Its name derives from the geographic analogy of a landscape being flooded by water.

General Intuition: The Flooding Process

Imagine piercing holes at each local minimum of the surface and slowly immersing the entire landscape into a lake. As the water level rises, catchment basins begin to fill up. Eventually, the water from two different basins will meet. To prevent them from merging, a "dam" (the **Watershed line**) is built at the point of contact. At the end of the process, the image is partitioned into several catchment basins, each associated with a unique local minimum.

Formal Algorithmic Approach

Given a PL function f and the filtration $K_0 \subset K_1 \subset \cdots \subset K_n$:

1. **Sorting:** Vertices are sorted in non-decreasing order of $f(v)$.
2. **Lower Star Extension:** For each vertex v_i, consider $St_-(v_i)$. If v_i is a local minimum, a new label is created: **birth** of a 0-cycle in H_0.
3. **Propagation:** If v_i is not a minimum, it is assigned the label of its neighbours in the lower link.
4. **The Merge Problem (Saddles):** If the lower link of v_i contains vertices with different labels, v_i is a **saddle point**.

Addressing Over-segmentation via Homology

The main drawback of the standard Watershed is that every local minimum, even one caused by microscopic noise, creates a basin. This is where **Persistent Homology** becomes essential [1]:

- Each basin is a class in the 0-th homology group H_0.
- When two basins meet at a saddle point s, the "younger" basin dies and merges into the "older" one.
- The **Persistence** $P = f(s) - f(v_{\min})$ measures the depth of the basin.

By setting a threshold τ, we only preserve segments where $P > \tau$. This is equivalent to performing a **topological simplification** of the Morse function before running the Watershed.

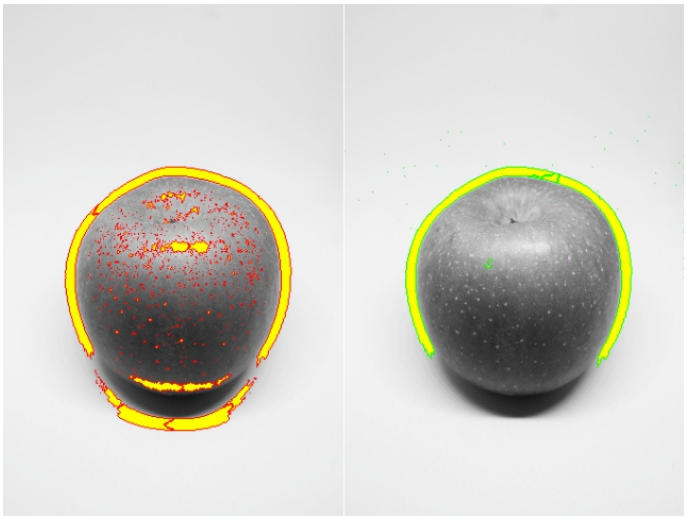


Figure 11: Left: over-segmentation due to noise. Right: Watershed guided by persistent homology filtering.

Implementation in R

The code uses `imager` to load images; every other step (smoothing, gradient, Union-Find, watershed, persistence) is written in base R.

Pipeline Overview

1. **Loading and normalisation.** The image is read via `imager::load.image`, converted to a greyscale matrix, and normalised to $[0, 1]$. An optional inversion ($f \leftarrow 1 - f$) makes bright objects (e.g. fluorescence microscopy) into valleys.
2. **Gaussian smoothing.** A separable Gaussian kernel suppresses JPEG artefacts and high-frequency noise.
3. **Construction of f.** Either the smoothed intensity or the Sobel gradient magnitude is used as the topographic surface.
4. **Persistence computation.** A scan in increasing order of f detects local minima (births) and saddle points (deaths) via Union-Find.
5. **Seeded watershed.** Only minima with persistence $\geq \tau$ are kept as seeds; a priority-queue BFS assigns every pixel to the nearest surviving basin.

Gaussian Smoothing

Given σ, the 1-D kernel is truncated at radius $r = \lceil 3\sigma \rceil$:

$$k_i = \frac{\exp\!\left(-i^2/2\sigma^2\right)}{\sum_{j} \exp(-j^2/2\sigma^2)}, \quad i \in \{-r, \dots, r\}.$$

```r
gauss_kernel1d <- function(sigma) {
  r   <- ceiling(3 * sigma)
  idx <- seq(-r, r)
  k   <- exp(-idx^2 / (2 * sigma^2))
  k / sum(k)
}

conv_rows <- function(mat, k) {
  nr <- nrow(mat);  nc <- ncol(mat)
  r  <- (length(k) - 1L) %/% 2L
  pad <- cbind(mat[, rep(1L, r)], mat, mat[, rep(nc, r)])
  out <- matrix(0, nr, nc)
  for (i in seq_along(k))
    out <- out + k[i] * pad[, i:(i + nc - 1L)]
  out
}

gaussian_blur <- function(mat, sigma) {
  k <- gauss_kernel1d(sigma)
  t(conv_rows(t(conv_rows(mat, k)), k))
}
```

Listing 1: Separable Gaussian blur in R.

Sobel Gradient

$$G_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}, \qquad G_y = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}, \qquad \|\nabla f\| = \sqrt{G_x^2 + G_y^2}.$$

```r
sobel_magnitude <- function(mat) {
  nr <- nrow(mat);  nc <- ncol(mat)
  p <- matrix(0, nr + 2L, nc + 2L)
  p[2:(nr+1L), 2:(nc+1L)] <- mat
  p[1L, ] <- p[2L, ];  p[nr+2L, ] <- p[nr+1L, ]
  p[, 1L] <- p[, 2L];  p[, nc+2L] <- p[, nc+1L]
  r <- 2:(nr+1L);  c <- 2:(nc+1L)
  gx <- -p[r-1,c-1] + p[r-1,c+1] - 2*p[r,c-1] + 2*p[r,c+1] -
         p[r+1,c-1] + p[r+1,c+1]
  gy <- -p[r-1,c-1] - 2*p[r-1,c] - p[r-1,c+1] +
         p[r+1,c-1] + 2*p[r+1,c] + p[r+1,c+1]
  matrix(sqrt(gx^2 + gy^2), nr, nc)
}
```

Listing 2: Sobel gradient magnitude in R.

Union-Find and Persistence Pairs

Role of the Union-Find structure

As the algorithm scans pixels in increasing order of f, it must answer two questions at each step: *which basin does each already-visited neighbour belong to?* and *are two neighbouring pixels in the same basin or in different ones?*

A simpler approach would store a label matrix and trace labels back to their origin at every merge, but this degenerates to $O(N^2)$ in the worst case. The **Union-Find** (also called disjoint-set) data structure solves both questions in near-constant amortised time by maintaining a forest of trees, one per basin, where the *root* of each tree is the canonical representative of that basin.

- **Path compression.** When searching for the root of a node, every node along the path is re-attached directly to the root, flattening the tree for all future lookups:

```r
uf_find <- function(uf, x) {
  root <- x
  while (uf$parent[root] != root) root <- uf$parent[root]
  # re-attach every node on the path directly to root
  while (uf$parent[x] != root) {
    nxt <- uf$parent[x]; uf$parent[x] <- root; x <- nxt
  }
  root
}
```

Listing 3: Path compression in `uf_find`.

- **Union by rank.** When merging two trees, the shorter one is attached to the taller one, preventing the structure from degenerating into a linear chain.

In addition to the parent and rank arrays, each component stores the **birth value** of its minimum, which is the key quantity needed to compute persistence pairs. The three operations performed during the main scan are summarised in Table 1.

Table 1: The three cases encountered when processing pixel v and the corresponding Union-Find operation.

Situation	**Topological event**	**UF operation**
No labelled neighbour	Birth: new local minimum	`uf_new_component`$(f(v))$
Neighbours, same basin	Regular propagation	`uf_find` (label lookup)
Neighbours, different basins	Death: saddle point	`uf_unite` (merge + record pair)

Birth (local minimum). A pixel v is a local minimum if none of its 8-connected neighbours has a smaller f value. It creates a new component with $\text{birth}(v) = f(v)$.

Death (saddle point). If at least two distinct components are adjacent to v, it is a saddle. Following the Elder Rule, the *younger* component (born at a higher f value) dies and is absorbed by the *older* one. The persistence pair recorded is:

$$(\text{birth}_{\text{dying}},\ f(v)), \qquad p = f(v) - \text{birth}_{\text{dying}}.$$

```r
for (idx in order(fmat)) {
  nbrs       <- neighbors8(idx, nr, nc)
  labeled_nb <- nbrs[labels[nbrs] > 0L]

  if (length(labeled_nb) == 0L) {
    cid <- uf_new_component(uf, fmat[idx])
    labels[idx] <- cid
    seed_pixel  <- c(seed_pixel, idx)
  } else {
    roots  <- unique(vapply(labels[labeled_nb],
                            function(l) uf_find(uf, l), integer(1L)))
    roots  <- unique(vapply(roots,
                            function(r) uf_find(uf, r), integer(1L)))
    births <- vapply(roots, function(r) uf$birth[r], numeric(1L))
    main_root   <- roots[which.min(births)]
    labels[idx] <- main_root
    for (r in roots) {
      r1 <- uf_find(uf, main_root);  r2 <- uf_find(uf, r)
      if (r1 != r2) {
        pair <- uf_unite(uf, r1, r2, fmat[idx])
        if (!is.null(pair))
          pairs_list[[n_pairs <- n_pairs + 1L]] <- pair
      }
    }
  }
}
```

Listing 4: Main watershed loop with persistence computation.

Topological Simplification and Seeded Watershed

After the full watershed scan, the algorithm holds a list of all local minima (`seed_pixel`) and all persistence pairs. A **seed** is the specific pixel that corresponds to a surviving local minimum, i.e. a minimum whose persistence $p \geq \tau$.

The seeded watershed separates two decisions that are conflated in the standard algorithm:

1. **How many regions to create**: controlled by the persistence threshold τ: only minima with $p \geq \tau$ become seeds.
2. **How to draw the boundaries**: controlled by the geometry of f: a priority-queue BFS expands each seed in order of increasing intensity, in order to let boundaries fall along the ridges of the surface.

Seed selection and BFS expansion

We have to remove from the full list of minima those whose `dying_id` appears in a low-persistence pair.

```
cancelled     <- prs$dying_id[prs$persistence < tau]
surviving_ids <- setdiff(seq_len(n_minima), cancelled)
seeds         <- seed_pixel[surviving_ids]
```

Listing 5: Simplification and seed selection.

Each surviving seed is then inserted into a min-heap (priority queue) ordered by f. At every step the pixel with the lowest intensity is extracted, its unlabelled 8-connected neighbours are assigned the same label, and they are pushed into the queue. This is equivalent to simulating the flooding process starting only from the topologically significant minima: the boundary between any two regions is the set of pixels where two flood fronts, originating from different seeds, first meet creating the watershed line.

Parameters

Parameter	Default	Description
`INVERT`	`TRUE`	Invert intensities. Use `TRUE` for bright objects on dark background.
`SIGMA`	`1.5`	Global Gaussian smoothing σ.
`SIGMA_GRADIENT`	`0`	Additional smoothing before Sobel (only with `USE_GRADIENT=TRUE`).
`USE_GRADIENT`	`FALSE`	If `TRUE`, f is the Sobel magnitude; otherwise smoothed intensity.
`TAU_QUANTILE`	`0.95`	Quantile of the persistence distribution used as τ.
`RESIZE_TO`	`150`	Maximum side length after proportional downscaling.

Settings

Image type	`INVERT`	`USE_GRADIENT`	`SIGMA`	`TAU_QUANTILE`
Fluorescence microscopy	TRUE	FALSE	1.5	0.95
Natural photo	FALSE	TRUE	0	0.95
Dark circles on light bg	FALSE	FALSE	1.5	0.95
Synthetic / clean image	FALSE	FALSE	0	0.95

Output and Visualisation

The function returns a list with: `original`, `gradient`, `pairs`, `tau`, `n_segments`, and `labels`. Six panels are produced.

Experimental Results

The implementation was tested on two cases with different characteristics: a controlled synthetic image, where the ground truth is known exactly, and a set of white circles on a black background, representing cells.

Case 1 — Synthetic Image

The synthetic image is generated directly in R as a 100×130 pixel matrix. It consists of three deep Gaussian wells (high-persistence basins) and one shallow well (low-persistence basin), all embedded in a flat background with additive Gaussian noise ($\sigma_{\text{noise}} = 0.035$):

$f(x,y) = 1 - 0.85\, e^{-\frac{(x-0.20)^2+(y-0.28)^2}{0.011}} - 0.75\, e^{-\frac{(x-0.72)^2+(y-0.32)^2}{0.010}} - 0.60\, e^{-\frac{(x-0.45)^2+(y-0.75)^2}{0.009}} - 0.25\, e^{-\frac{(x-0.78)^2+(y-0.80)^2}{0.005}} + \varepsilon$, where $\varepsilon \sim \mathcal{N}(0,\, 0.035^2)$ pixel-wise.

The parameters used are:

$$\texttt{INVERT = FALSE}, \quad \texttt{USE_GRADIENT = FALSE}, \quad \sigma = 0, \quad \texttt{TAU_QUANTILE = 0.95}$$

Since the image is noise-free by construction and the blobs are darker than the background, neither inversion or smoothing are required.

The persistence diagram shows a separation between the three high-persistence points and the cluster of low-persistence points near the diagonal. After applying the $\tau_{0.95}$ threshold, exactly three regions survive, matching the three dominant Gaussian wells.

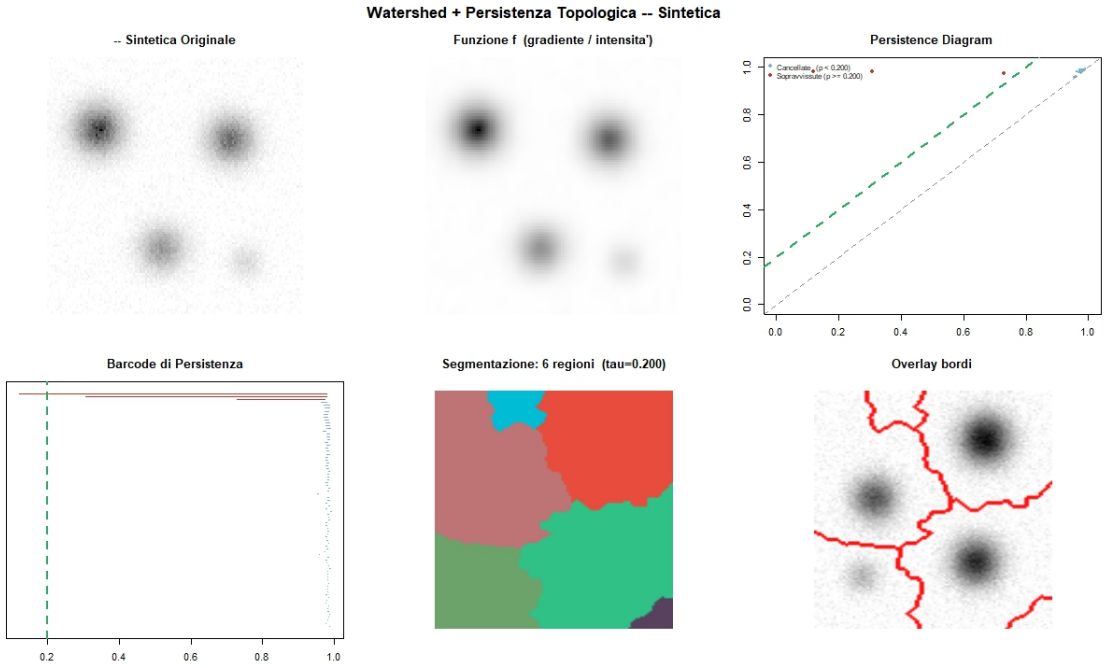


Figure 12: Results on the synthetic image (`INVERT = FALSE`, $\sigma = 0$, `TAU_QUANTILE = 0.95`). Top row: original image, function f, persistence diagram. Bottom row: barcode, segmentation map, boundary overlay. The three high-persistence wells are correctly identified; the shallow fourth well is cancelled by the persistence filter.

Case 2 — White Circles on Black Background

The second test image consists of 34 white circles of different radii arranged on a black background (680×510 pixels, saved as JPEG). This setup mimics the appearance of fluorescence microscopy images, where cell nuclei or other organelles appear as bright structures against a dark field.

Because the objects of interest are *bright* (high intensity), while the watershed algorithm finds *minima*, the image must be inverted before processing ($f \leftarrow 1 - f$). After inversion, each circle becomes dark (a local minimum), and the black background becomes white. The parameters used are:

$$\texttt{INVERT = TRUE}, \quad \texttt{USE_GRADIENT = FALSE}, \quad \sigma = 3.0, \quad \texttt{TAU_QUANTILE = 0.99}$$

The high smoothing ($\sigma = 3.0$) is necessary to suppress JPEG compression artefacts. The threshold (`TAU_QUANTILE = 0.99`) retains only the 1% most persistent components.

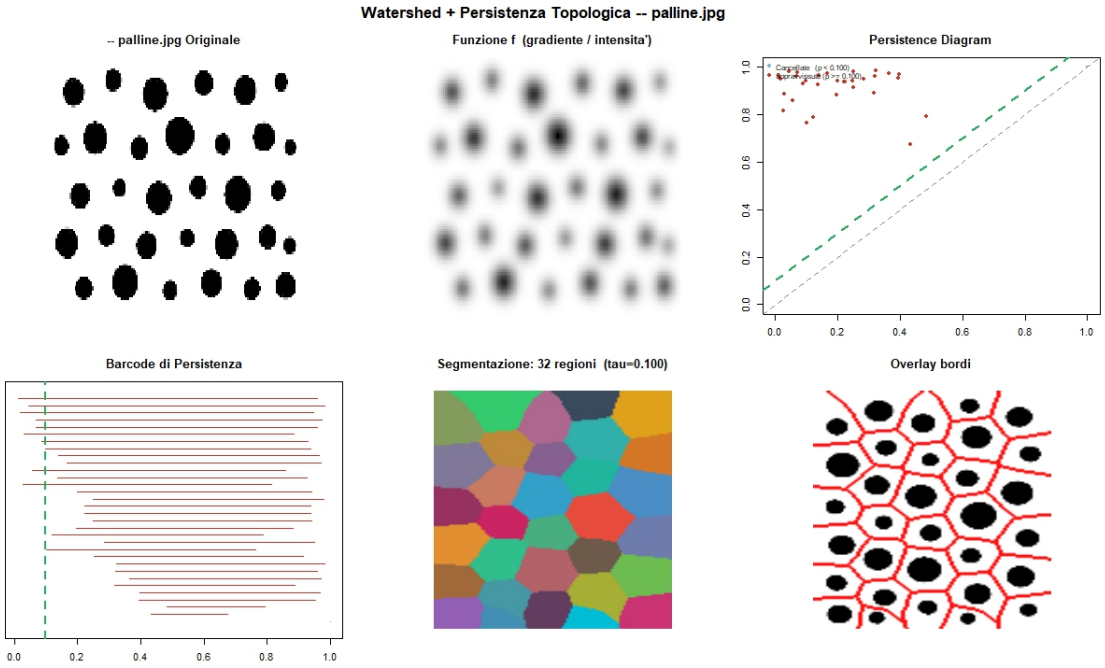


Figure 13: Results on the white-blobs image (`INVERT = TRUE`, $\sigma = 3.0$, `TAU_QUANTILE = 0.99`). Top row: original image, function f after inversion and smoothing, persistence diagram. Bottom row: barcode, segmentation map, boundary overlay. Each surviving region corresponds to one circular blob.

Comparison and Discussion

Table 2: Comparison between the two experimental cases.

	Synthetic image	**White blobs (JPEG)**
Image size	100×130 px	680×510 px (resized)
Objects of interest	Dark Gaussian wells	Bright circles
`INVERT`	FALSE	TRUE
`SIGMA`	0	3.0
`USE_GRADIENT`	FALSE	FALSE
`TAU_QUANTILE`	0.95	0.99
Expected regions	3 (+ 1 shallow)	34 circles
Key challenge	Shallow false basin	JPEG compression artefacts
Solution	Persistence filter	High σ + aggressive τ

The two cases illustrate the two main problems in watershed segmentation. In the synthetic case, the challenge is topological: the algorithm must distinguish a genuine basin from a spurious one of similar spatial extent but lower depth.

In the blob case, the challenge is numerical: JPEG compression introduces high-frequency intensity fluctuations that create hundreds of false minima within each circle.

References

[1] H. Edelsbrunner and J. Harer, *Computational Topology: An Introduction*. American Mathematical Society, Providence, RI, 2010.

[2] G. Naldi, L. Pareschi and G. Russo, *Matematica Numerica: Esercizi, Laboratori e Progetti*. Springer, Milano, 2016.

[3] The MathWorks, Inc., *Image Processing Toolbox — `edge` function (Sobel method)*, MATLAB R2024a Documentation. Available at: https://www.mathworks.com/help/images/ref/edge.html (accessed March 2026).

[4] CP-Algorithms, *Disjoint Set Union (Union-Find)*. Available at: https://cp-algorithms.com/data_structures/disjoint_set_union.html (accessed March 2026).

[5] C. Maria, J.-D. Boissonnat, M. Glisse, M. Yvinec, The Gudhi Library in Python: Simplicial Complexes and Persistent Homology. ICMS 2014, LNCS vol. 8592, Springer.